

Course Code: CS1107

Name of the Programme: BCA (Hons.)

Name of the Course: Digital Logic

Semester: I

Course Credit	No. of Hours Per Week (L+ T + P)	Total No. of Teaching Hours
3	1:1:2	60
Course Objectives:		
a) Introduce the fundamental principles of digital logic design and its role in computing systems.		
b) Develop the ability to analyze and design basic digital circuits using combinational and sequential logic.		
c) Explore the structure, function, and interaction of core computer architecture components.		
d) Apply theoretical knowledge to practical scenarios, laying a strong foundation for advanced studies in computer systems and engineering.		

Course outline (Syllabus of the course)

Module - 1 (Digital Systems and Binary Numbers, Boolean Algebra and Logic Gates)	No. of Hours
	3 Hrs.
Digital systems, binary numbers, number base conversions, octal and hexadecimal numbers, complements, signed binary numbers, binary codes, error detection and error correction codes.	
Basic definitions, axiomatic definition of Boolean algebra, basic theorems and properties of Boolean algebra, Boolean functions, canonical and standard forms.	
Module - 2 (Gate Level Minimization)	No. of Hours
	3 Hrs.
The k-map method, four-variable map, five-variable map, product of sums simplification, don't-care conditions, NAND and NOR implementation	
Module - 3 (Combinational Circuits)	No. of Hours
	3 Hrs.
Design procedure, Binary Adder, Binary Subtractor, Decoders, Encoders, Multiplexers, and Demultiplexers.	
Module - 4 (Synchronous Sequential Logic, Registers and Counters)	No. of Hours

	3 Hrs.
Sequential circuits, latches, flip-flops Registers, shift registers, ripple counters	
Module - 5 (Memory and Programmable Logic)	No. of Hours
	3 Hrs.
Introduction, Random access memory, memory decoding, read only memory, programmable logic array, programmable array logic, sequential programmable devices.	

Course Outcomes:
a) Understand binary number systems, codes, and error detection/correction techniques in digital systems.
b) Apply Boolean algebra to simplify logic expressions and design circuits using logic gates and standard forms.
c) Analyze and design combinational and sequential circuits such as adders, multiplexers, and counters for practical applications.
d) Evaluate the role and functionality of memory and programmable logic devices like RAM, ROM, and PLDs in digital systems.

Textbooks	
1	Mano, M. M., & Ciletti, M. D. (2017). Digital Design (6th ed.). Pearson. ISBN: 978-0134549897
2	Roth, C. H., Kinney, L. L., & John, C. H. (2016). Fundamentals of Logic Design (7th ed.). Cengage Learning. ISBN: 978-1305635142

Reference Books	
1	Rao, C. V. S. (2009). <i>Switching and logic design</i> (3rd ed.). Pearson Education. ISBN: 9978-81-317-0183-6
2	Wakerly, J. F. (2018). Digital Design: Principles and Practices (5th ed.). Pearson. ISBN: 978-0134460093

3	Harris, D. M., & Harris, S. L. (2021). Digital Design and Computer Architecture (3rd ed.). Morgan Kaufmann. ISBN: 978-0128200650
---	--

Reference Links	
1	https://www.geeksforgeeks.org/digital-electronics-logic-design-tutorials/
2	https://www.javatpoint.com/digital-electronics
3	https://www.tutorialspoint.com/digital-electronics/index.htm

Tutorial Component	
1.	Perform number base conversions between binary, octal, decimal, and hexadecimal systems.
2.	Explain the concept of complements (1's and 2's) and perform arithmetic operations using signed binary numbers.
3.	Encode given data using binary codes and design an error detection/correction mechanism using Hamming codes.
4.	Apply the axiomatic definition of Boolean algebra to prove basic theorems and simplify Boolean expressions.
5.	Represent Boolean functions in canonical forms (SOP and POS) and standard forms.
6.	Simplify a given Boolean function using Karnaugh Maps (K-map) for up to five variables and implement the simplified expression.
7.	Design and implement a Boolean expression using NAND and NOR gates only.
8.	Design a combinational circuit for a binary adder and implement it using logic gates.
9.	Design a combinational circuit for a binary subtractor and explain its operation.
10.	Create encoder, decoder, multiplexer, and demultiplexer circuits and explain their applications in digital systems.
11.	Design a sequential circuit using flip-flops and explain the differences between latches and flip-flops.
12.	Design and implement shift registers for specific bit-length operations (e.g., serial-in serial-out, serial-in parallel-out).
13.	Construct a ripple counter circuit and analyze its timing diagram.
14.	Explore the structure of memory devices, explain RAM and ROM functionality, and design a memory decoding circuit.
15.	Explore the structure of memory devices, explain RAM and ROM functionality, and design a memory decoding circuit.

Laboratory Component	
1	Verification of truth tables of the following Logic gates Two input (i) OR (ii) AND (iii) NOR (iv) NAND (v) Exclusive-OR (vi) Exclusive-NOR
2	Design a simple combinational circuit with four variables and obtain minimal expression and verify the truth table using Digital Trainer Kit.
3	Verification of functional table of 3 to 8-line Decoder /De-Multiplexer.
4	4 variable logic function verification using 8 to1 multiplexer.
5	Design a full adder circuit and verify its functional table.
6	Verification of functional tables of (i) JK Edge triggered Flip-Flop (ii) JK Master Slave Flip-Flop (iii) D Flip-Flop
7	Design a four-bit ring counter using D Flip-Flops/JK Flip Flop and verify output
8	Design a four bit Johnson's counter using D Flip-Flops/JK Flip Flops and verify output
9	Verify the operation of 4-bit Universal Shift Register for different Modes of operation.
10	Draw the circuit diagram of MOD-8 ripple counter and construct a circuit using T-Flip- Flops and Test It with a low frequency clock and sketch the output waveforms.
11	Design MOD-8 synchronous counter using T Flip-Flop and verify the result and sketch the output waveforms.
12	(a) Draw the circuit diagram of a single bit comparator and test the output (b) Construct 7 Segment Display Circuit Using Decoder and 7 Segment LED and test it.